

SP 800-232, Ascon-based lightweight cryptography standards for constrained devices



As part of the course 'Applied Cryptography and Cryptanalysis'

VITION, ANDRIANA
KALAITZIDIS IOANNIS
ELENI TSERMENTSELI

SUPERVISING PROFESSOR:
DR PANAGIOTIS RIZOMYLIOTIS



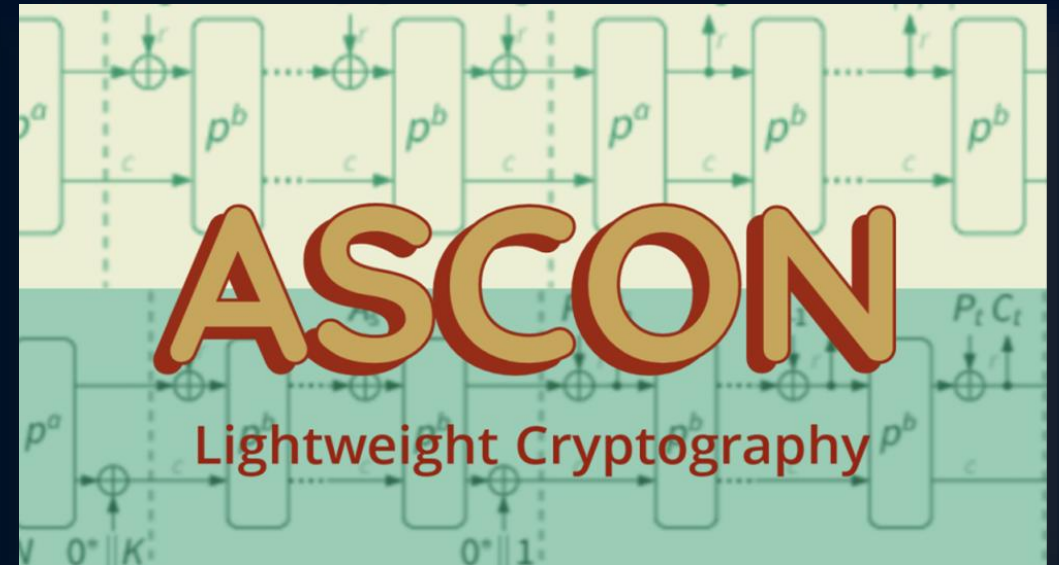
What is SP 800-232?

NIST SP 800-232 is published by the National Institute of Standards and Technology and is now the official US (and global) standard **for IoT security**. Its name indicates that it belongs to the Special Publication (SP) category, which includes technical guidelines and recommendations. Specifically, it forms part of the 800 series, which is exclusively dedicated to Information Systems Security, with the unique serial number 232 identifying this particular document [1].

It was designed by a team of researchers at Graz University of Technology (TU Graz) in Austria (Christoph Dobraunig, Maria Eichlseder, Florian Mendel, Martin Schl  ffer). **It won the CAESAR competition** (2019) for lightweight cryptography and subsequently the **NIST competition** (2023), prevailing over 56 other entries.

What does it specify?

The standard establishes the **Ascon family of algorithms** as the official solution for **Lightweight Cryptography** for constrained devices [5].



What is AES?



What is the aim of SP 800-232?

It offers cryptographic solutions (encryption and hashing) specifically designed to be efficient in situations where the standard AES may be 'resource-intensive' [2].

AES (Advanced Encryption Standard) is the most widely used and accepted standard for symmetric encryption worldwide today [3]. It is the 'conventional' encryption standard used everywhere today (in e-banking, HTTPS, secure files, high-performance computers, smartphones, etc.) [4].

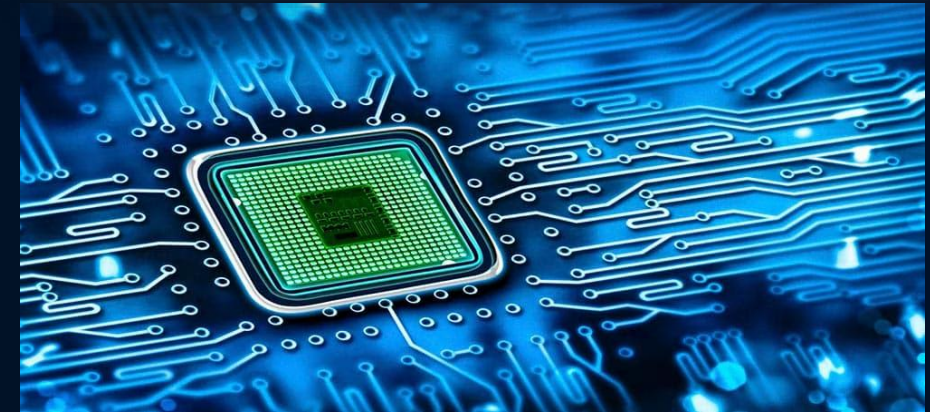


What devices is the SP 800-232 designed for?

- **IoT (Internet of Things) devices:** All 'smart' devices connected to the internet, such as IoT sensors that collect data from the environment.
- **Embedded systems:** Small computers embedded in other machines, such as the control unit of a washing machine, a car's electronics or industrial control systems.
- **Low-power sensors:** Devices that are usually battery-powered, such as RFID tags, low-power medical devices (e.g. pacemakers) and sensors for recording temperature or movement.

Common feature: They have **minimal memory**, **low processing power** and **limited battery life**.

Due to the high demands of **AES**, which render it **inefficient** in such conditions, **Ascon** emerges as **the optimal choice** for these environments [6][7].



Comparison of AES and Ascon

Feature	AES	Ascon
Scope of Application	General-purpose and high-performance (conventional standards) [11]	Devices with Limited Resources (IoT, Embedded Systems, Sensors) [8]
Resource Requirements	High (Resource-Intensive). May not perform optimally on small devices [9], [11]	Low (Lightweight). Designed for efficiency in resource-constrained environments [10]
Architecture	Block Cipher (Classical encryption) [11]	Permutation-based [8], [12]
Decryption Process	Requires implementation of the inverse process for decryption.	Inverse-free: There is no need to implement the inverse permutations, saving space and reducing complexity [13]

The Ascon Family of Algorithms

The Ascon algorithm family is a set of **lightweight cryptographic algorithms** specifically designed to operate efficiently on devices with limited resources (such as IoT and embedded systems) [8]. It comprises four main members [14][22]:

- **Ascon-AEAD128**: Provides Authenticated Encryption, simultaneously ensuring data confidentiality and verifying that the data has not been tampered with.
- **Ascon-Hash256**: Functions as **a hash function** (similar to SHA-256 but more lightweight), producing a **fixed 256-bit 'fingerprint'**.
- **Ascon-XOF128**: This is an **Extensible Output Function (XOF)** that allows the user to **choose the output bit length arbitrarily**.
- **Ascon-CXOF128**: A **variant of the XOF** that offers **customisation options** for **secure** domain separation.

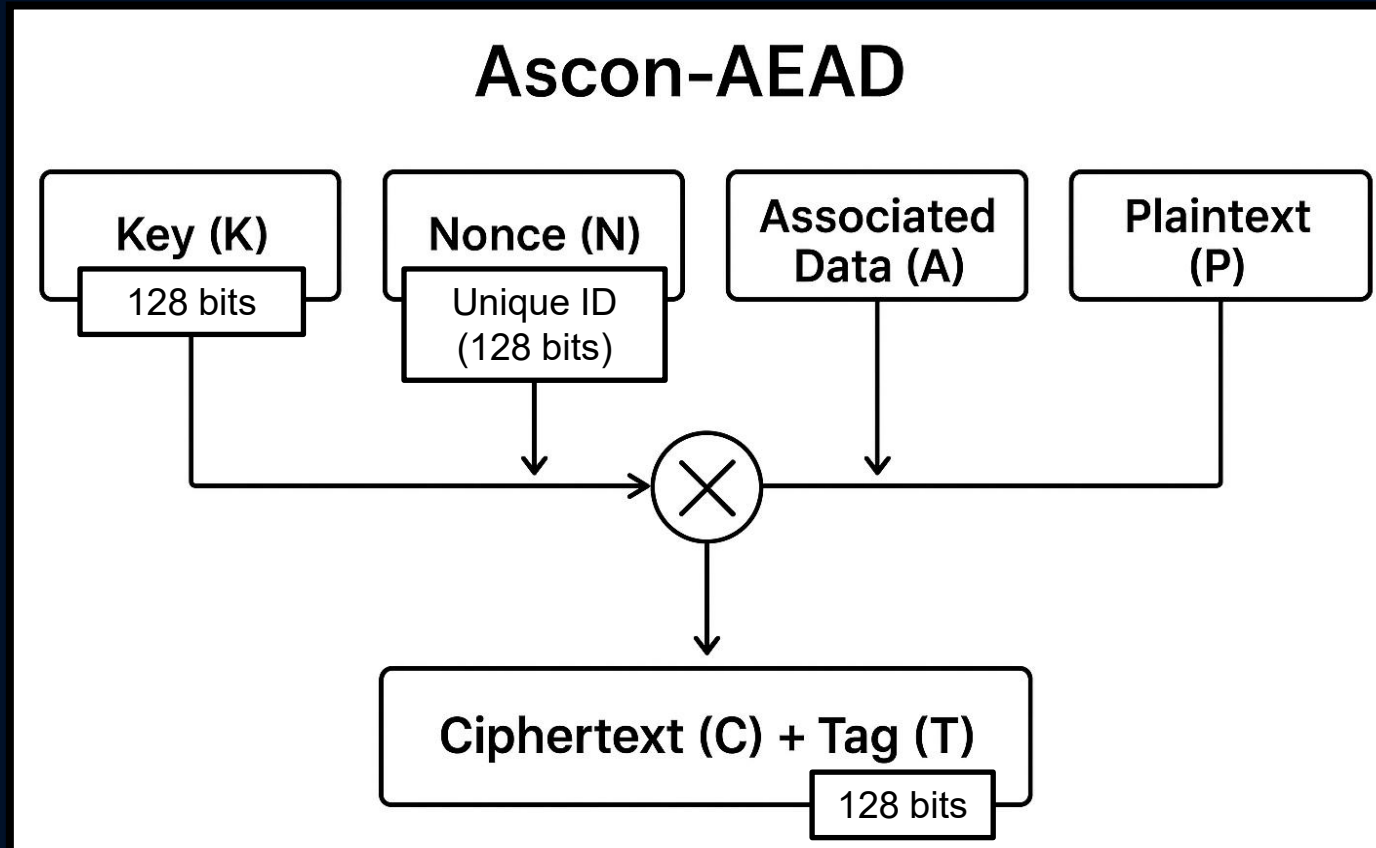
Ascon-AEAD128 Algorithm (1/4)

1. Authenticated Encryption (AEAD):

The **Ascon-AEAD128** algorithm is at the heart of the standard. It does not merely encrypt the data to ensure it remains confidential, but also guarantees its integrity. It takes a **secret key**, a **unique number (nonce)** and **the data**, and produces the **ciphertext** together with an **authentication tag**. If someone tampers with the encrypted data, the tag will not match during decryption and the device will reject the message as invalid [15].

AEAD (Authenticated Encryption with Associated Data) functions as an 'all-in-one' mechanism. As well as encrypting sensitive data, it simultaneously 'digitally locks' the entire packet (including any unencrypted data, such as headers), producing a **Tag**. If a hacker alters even a single bit in the packet, the Tag will not be verified and the device will reject the message immediately [16].

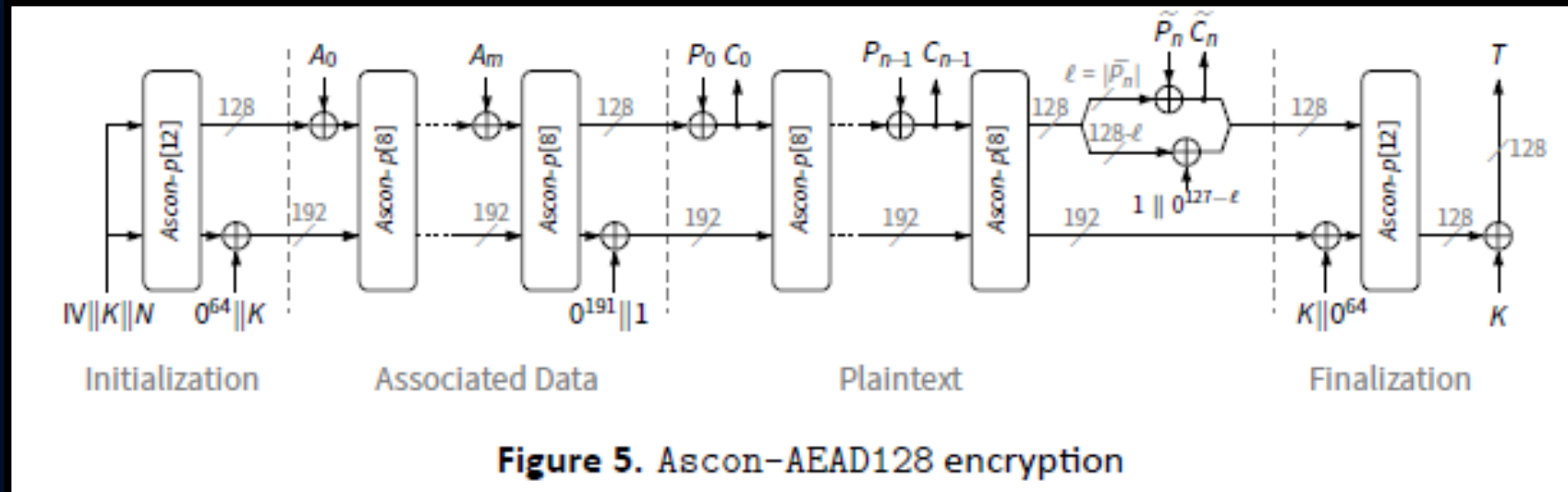
Algorithm flowchart (2/4)



Parameters for Ascon authenticated encryption

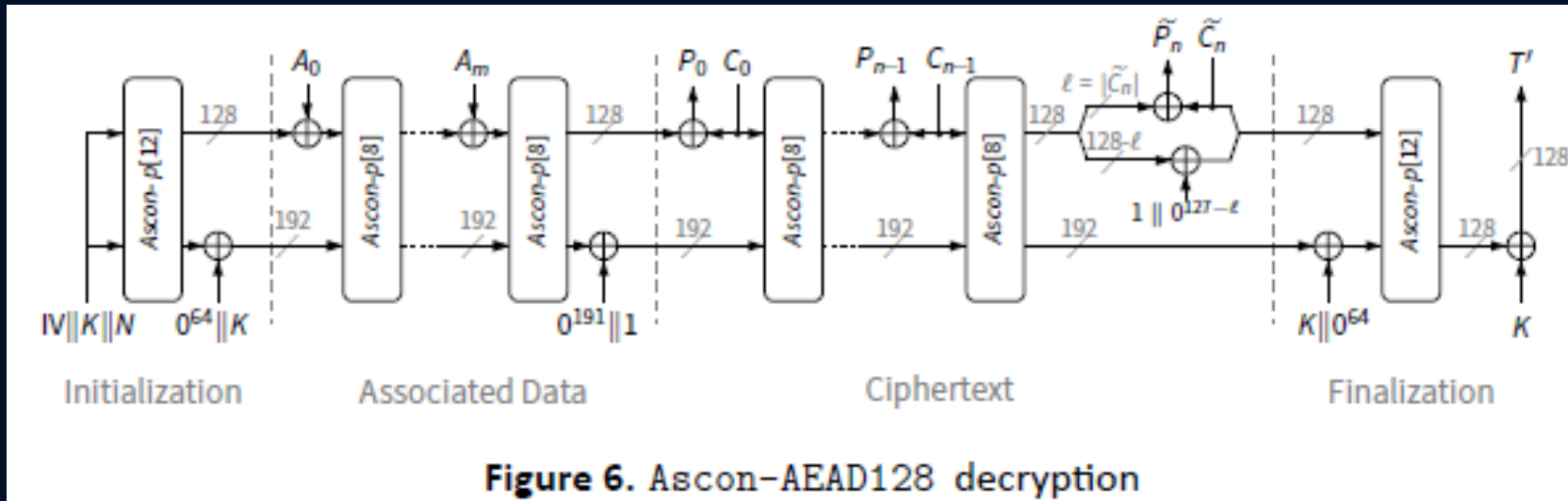
Cipher	Bit size of					Rounds	
	key	nonce	tag	rate	capacity	$p[12]$	$p[8]$
Ascon-AEAD128	128	128	128	128	192	12	8

Encryption Process (3/4)



- 1. Initialisation** [38]:
 - The Key (K), the Nonce (N) and the IV constant are loaded.
 - The first strong permutation (12 rounds) is performed to prepare state 2.
- 2. Processing of Associated Data** [39]:
 - The associated data (AD) is mixed into the state to ensure its authenticity, without being encrypted.
- 3. Encryption (Plaintext Processing)** [40]:
 - The plaintext (P) is input and XORed with the state.
 - This immediately produces the ciphertext (C).
 - The state is continuously updated (8 rounds) after each block.
- 4. Finalisation** [41]:
 - The Key (K) is re-entered for security
 - The 128-bit Tag (T) is generated, which seals the message.

Decryption Process (4/4)



- 1. Initialisation & Associated Data** [42]:
 - The process begins exactly as in encryption (K , N , IV and AD processing)
- 2. Ciphertext Processing** [43]:
 - The ciphertext (C) is input and XORed with the state.
 - The plaintext (P) is revealed.
 - The state is updated with the ciphertext for the next step
- 3. Tag Verification** [44]:
 - The algorithm calculates a new tag (T') based on the data it has received.
 - Comparison: It checks whether T' is the same as the tag (T) sent by the sender.

Result:

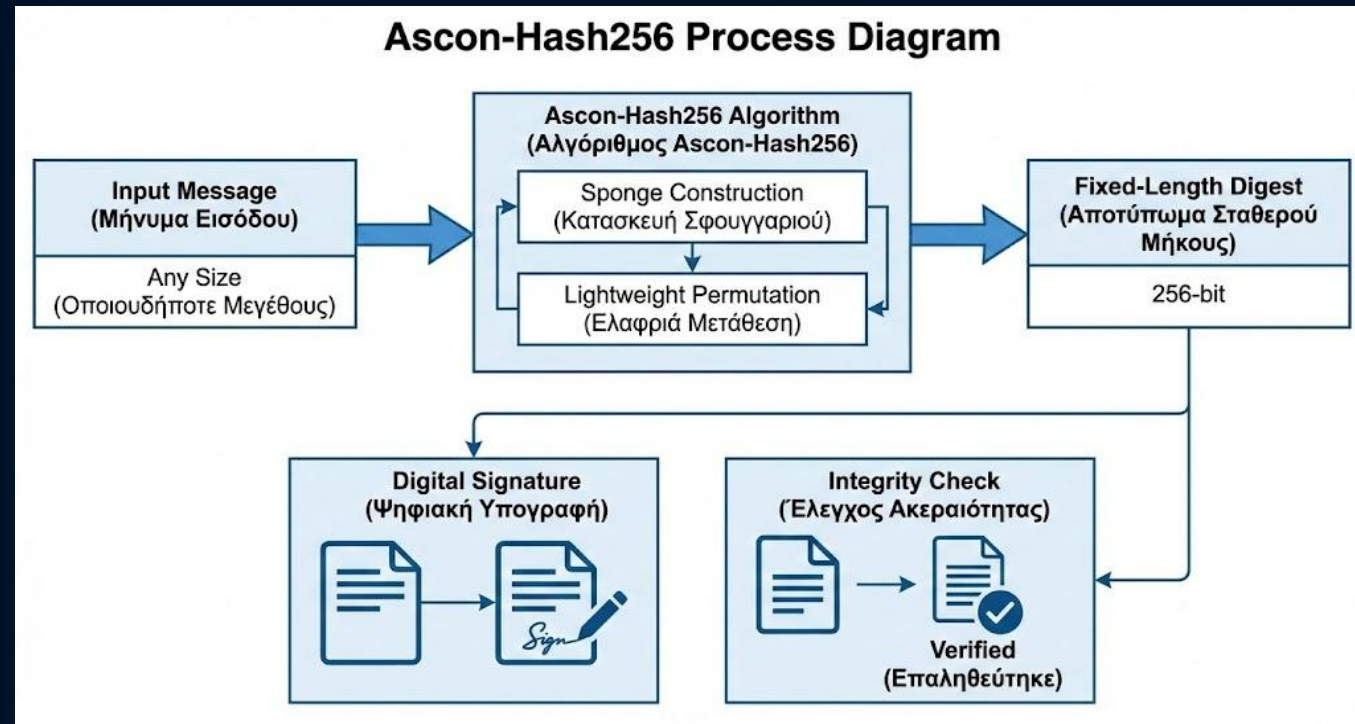
If they match $\rightarrow$ Returns the text (P).

If they do NOT match $\rightarrow$ Returns 'FAIL' and displays nothing.

Ascon-Hash256 algorithm

2. Hashing:

The **Ascon-Hash256** algorithm works in the same way as well-known hash functions (e.g. SHA-256), but is more lightweight [17]. It takes a message of any size and produces a unique 'digital fingerprint' (digest) of a fixed length of 256 bits [18]. This algorithm processes data more slowly (rate: 64 bits) compared to AEAD (rate: 128 bits) for security reasons relating to the hash structure [23]. It is used for digital signatures and file integrity checking.



Ascon-XOF128 & Ascon-CXOF128 Algorithms (1/2)

3. Extendable Output Functions (XOF):

The standard includes two XOF algorithms, which are similar to hashing but are more flexible because the output length is not fixed but is chosen by the user [19].

- **Ascon-XOF128**: The user specifies how many output bits they wish to be generated from the message [20].
- **Ascon-CXOF128 (Customised)**: A variant that allows the user to also enter a 'customisation string'. This helps to distinguish between different applications that use the same key or data (domain separation) [21].

Both algorithms support **a security strength of up to 128 bits**.

Ascon-XOF128 & Ascon-CXOF128 Algorithms (2/2)

Example (assuming the same number of bits):

Without Domain Separation (Ascon-XOF128):

The 'Email App' retrieves 'Secret123' → Produces the result XYZ.

The 'Wallet App' takes 'Secret123' → It also produces the result XYZ.

If a hacker steals the result from the Email app, they can use it to gain access to the Wallet app as well, because it is the same.

With Domain Separation (Ascon-CXOF):

The algorithm requests a 'tag' (customisation string) to determine its location.

The 'Email' app says: Code = 'Secret123' + Tag = 'EMAIL' → Result ABC.

The 'Wallet' application says: Code = 'Secret123' + Tag = 'WALLET' → Result 999.

The results are completely different. No one can use the data from one application in the other.

Ascon-Hashing Algorithms

Parameters for Ascon hashing				
Algorithm	Bit size of			Rounds
	hash output	rate	capacity	$p[12]$
Ascon-Hash256	256	64	256	12
Ascon-XOF128	arbitrary	64	256	12
Ascon-CXOF128	arbitrary	64	256	12

Ascon Algorithm Family

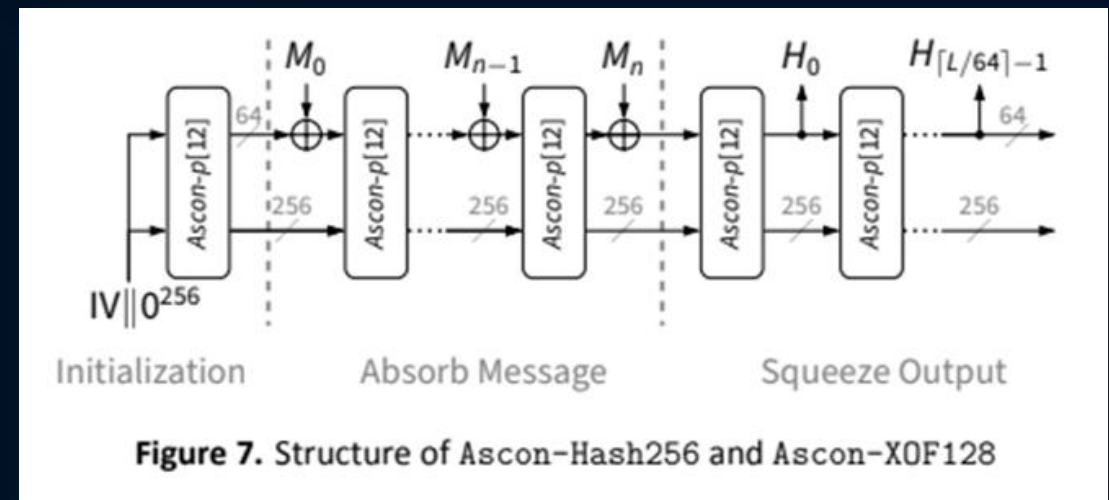
The Ascon family offers a comprehensive toolkit for the **security of small devices**, addressing both the need for **confidential communication** (via **AEAD**) and the need for data **integrity verification** (via **Hash/XOF**), all based on the same lightweight architecture [22].

Ascon Algorithm Architecture

All algorithms in the Ascon family are based on a **common architecture** known as **Sponge Construction**. It works exactly as the name suggests [23]:

- **Initialisation**: We start with an initial state.
- **Absorbing Phase**: The algorithm 'sucks in' the input data bit by bit and mixes it into the state.
- **Squeezing Phase**: The algorithm 'squeezes' the state to produce the result (the ciphertext, the tag or the hash).

However, to ensure that this mixing is secure between the absorption and squeezing phases, we need a **strong mixing mechanism**. This is achieved using **permutations** [25].



What are permutations?

The Ascon-p **permutations** form the **central** processing **core of the algorithm**, operating on an **internal state with** a total size **of 320 bits** [26]. To achieve optimal efficiency, this state is structurally **organised** into **five** separate **64-bit words** (S0 to S4), enabling extremely fast data processing [27].

Variations and Operating Cycles

Two basic permutation variants are used to achieve the optimal balance:

Ascon-p[12]: Runs the process 12 times (**slower, more secure**). Used for initialisation and encryption termination [28].

Ascon-p[8]: Executes the process 8 times (**faster**). It is used for processing intermediate data [28].

Although the standard mathematically defines the structure for 1 to 16 rounds, the Ascon 'family' uses exclusively 12 and 8 rounds to combine maximum security at the start and end with speed during data flow [29].

The **Ascon-Hash256** algorithm uses exclusively the 12-round permutation (Ascon-p[12]), whilst **Ascon-AEAD128** uses the 12- and 8-round permutations (Ascon-p[8]).

How secure are the Ascon algorithms (1/3)?

Function	Output size in bits	Security strength in bits		
		Collision	Preimage	2nd Preimage
Ascon-Hash256	256	128	128	128
Ascon-XOF128	L	$\min(L/2, 128)$	$\min(L, 128)$	$\min(L, 128)$
Ascon-CXOF128	L	$\min(L/2, 128)$	$\min(L, 128)$	$\min(L, 128)$

[37]

Ascon-Hash256:

- Provides a security strength of 128 bits (although the output is 256 bits).
- Resistant to all basic attacks (collision, preimage).

Ascon-XOF128 and CXOF128:

- The security is variable and depends on the output length (L) chosen by the user.
- Rule: Security increases as L increases, up to a maximum of 128 bits.
- For maximum security against collisions, an output of at least 256 bits is required.

Ascon-AEAD128:

- Confidentiality: 128 bits → No one can read the message without the key.
- Integrity: 128 bits → No one can alter the message (or the Nonce/Tag) without being detected [36].

How secure are the Ascon algorithms (2/3)?

Security	Security strength in bits	Total number of repetitions of a nonce
Confidentiality of plaintext	$\min\{128 - \log_2 u, \lambda\}$	$\leq 2^8$
Integrity of (N, A, C, T)	$\min\{128 - \log_2 u, \lambda\}$	$\leq 2^8$

Problem: On IoT devices, the same nonce may be used twice by mistake (e.g. after a restart).

Ascon's solution: Unlike other standards that 'collapse' immediately, Ascon provides a safety net [48]:

- **Repetition Limit:** It can withstand up to **(256) repetitions** of the same nonce without compromising security
- **Integrity Preservation:** Even in the event of an error, integrity remains high, preventing message forgery
- **Multi-User Security:** The '128-' model demonstrates that security remains robust even if there are millions of users on the network [49].

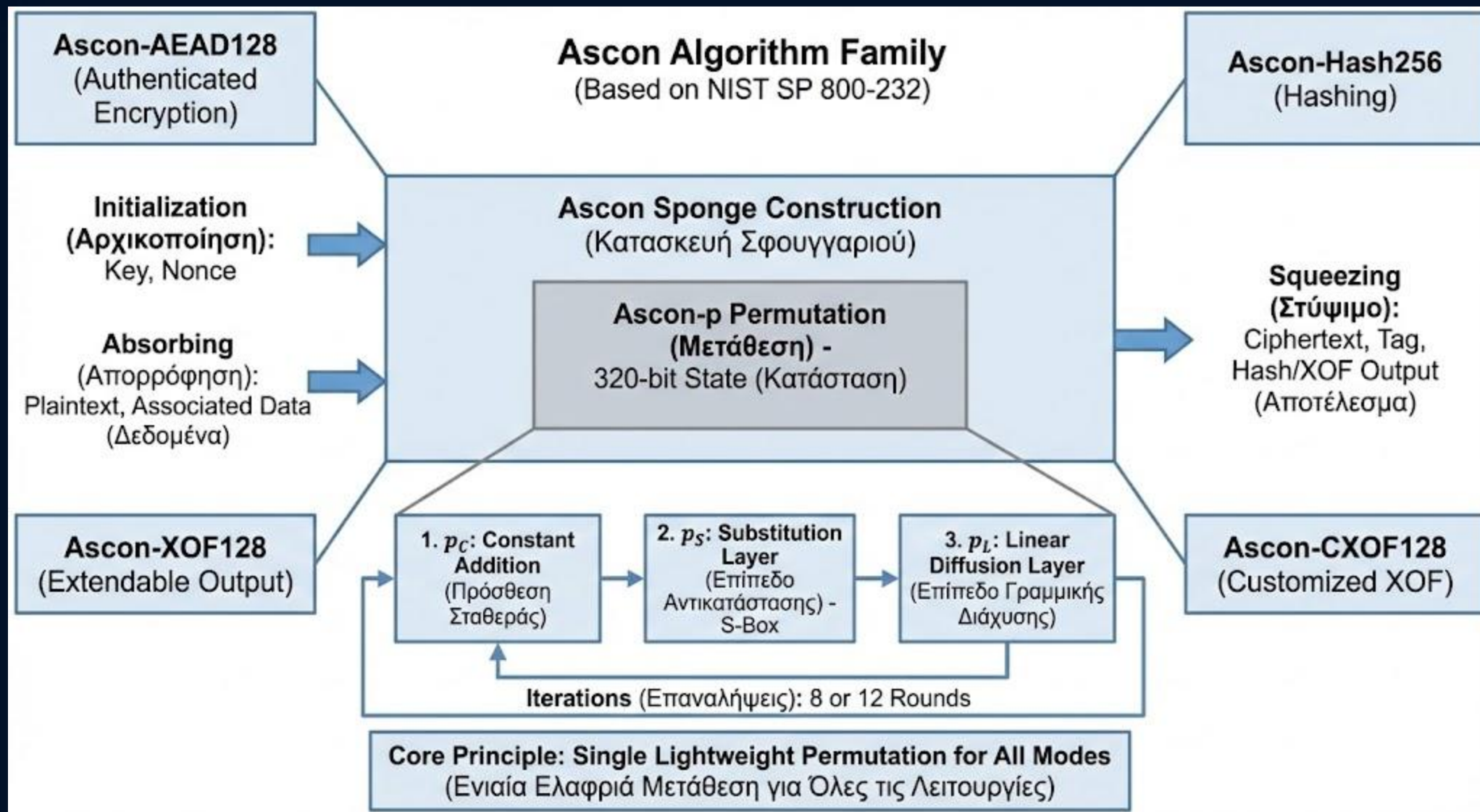
How secure are the Ascon (3/3) algorithms?



The **Ascon-AEAD128** algorithm guarantees strong 128-bit security, fully safeguarding both the **confidentiality** and **integrity** of the data [30]. At the same time, it is designed to demonstrate resilience even in scenarios involving unintentional repetition of the 'nonce', although its uniqueness remains a fundamental requirement for optimal operation [31]. Furthermore, recent research by Lefevre and Mennink demonstrates that, thanks to the unique '**Key** Blinding' technique (a strategy of repositioning the key at the end), Ascon remains secure even in cases where the internal state is recovered (State Recovery) by attackers [45][46]. This means that even if the intermediate state is leaked, the attacker cannot 'work their way through' the permutations to forge the message [47]. However, the standard imposes a clear operational restriction:

For any given key, the total volume of data processed (encryption and decryption) must not exceed bytes, making it mandatory to change the key once this limit is reached [32].

As for the **Hash** and **XOF** algorithms, because they do not use a secret key, they are not subject to this data volume restriction. Their security depends solely on the output length we choose (with a maximum of 128 bits).





Conclusion

In conclusion, the NIST SP 800-232 standard establishes Ascon as the official solution for IoT security, **filling the gap left by traditional standards in resource-constrained environments** [33]. This innovative architecture focuses on **resource efficiency**, using the **same shift register core** for all operations and **eliminating the need for separate decryption circuits (inverse-free)** [34]. This achieves an optimal balance, providing the necessary speed and efficiency for small devices, whilst maintaining strong 128-bit security without compromising data protection [35].

Thank you for your attention!



Bibliography (1/6)

Turan, M.S. (2025) Ascon-based lightweight cryptography standards for constrained devices:
<https://doi.org/10.6028/nist.sp.800-232>:

- [1]: "In February 2023, NIST announced the decision to standardise the Ascon family [8] for lightweight cryptography applications." (p.1)
- [2]: "The Ascon family is characterised by lightweight, permutation-based primitives and provides robust security... The family is designed to offer a viable alternative when the Advanced Encryption Standard (AES) may not perform optimally." (p.i)
- [3]: "National Institute of Standards and Technology... Advanced Encryption Standard (AES), FIPS 197." (p. 32)
- [4]: "The family has been developed to offer a viable alternative when the Advanced Encryption Standard (AES) may not perform optimally." (p.i), "NIST initiated the Lightweight Cryptography Standardisation Process... conventional cryptographic standards (e.g., AES-GCM...) may be resource-intensive." (p.1)
- [5]: "Ascon-Based Lightweight Cryptography Standards for Constrained Devices" (Title), "The Ascon family is designed to be efficient in constrained environments." (p.1)
- [6]: "...making it ideal for resource-constrained environments, such as Internet of Things (IoT) devices, embedded systems and low-power sensors. The family has been developed to offer a viable alternative when the Advanced Encryption Standard (AES) may not perform optimally." (p.i)
- [7]: "The family has been developed to offer a viable alternative when the Advanced Encryption Standard (AES) may not perform optimally." (p.i), "...making it ideal for resource-constrained environments, such as Internet of Things (IoT) devices, embedded systems and low-power sensors." (p.i), "NIST initiated the Lightweight... may be resource-intensive." (p.1)
- [8]: "The Ascon family is characterised by lightweight, permutation-based primitives... making it ideal for resource-constrained environments, such as Internet of Things (IoT) devices..." (p.5).
- [9]: "The family has been developed to offer a viable alternative when the Advanced Encryption Standard (AES) may not perform optimally." (p.5).
- [10]: "The Ascon family is designed to be efficient in constrained environments." (p.12)

Bibliography (2/6)

- [11]: "...conventional cryptographic standards (e.g., AES-GCM...) may be resource-intensive." (p.12)
- [12]: "Multiple functionalities. The same permutations are used to construct multiple functionalities..." (p.13)
- [13]: "Inverse-free. Since all members of the Ascon family only use the underlying permutations in the forward direction, there is no need to implement the inverse permutations." (Introduction, p.13)
- [14]: "The Ascon family is characterised by lightweight, permutation-based primitives... and low-power sensors." (p.i), "The Ascon family is designed to be efficient in constrained environments." (p.1)
- [15]: "Ascon-AEAD128 is a nonce-based AEAD scheme... single-key setting." (p.1), "Ascon-AEAD128.enc takes... and a 128-bit authentication tag T" (p.11), "Ascon-AEAD128.dec takes... outputs P if the tag is valid... otherwise fails" (p.11).
- [16]: "Authenticated Encryption with Associated Data (AEAD): Authenticated Encryption with Associated Data" (p.3), "Tag: A cryptographic checksum on data designed to reveal both accidental errors and intentional modification... knowledge of a secret key." (p.4), "Ascon-AEAD128.dec ... outputs P if the tag is valid... otherwise fails" (p.11).
- [17]: "Ascon-Hash256 is a cryptographic hash function that produces a 256-bit hash of the input messages, offering a security strength of 128 bits." (p.1)
- [18]: "Ascon-Hash256 takes a variable-length message M as input and produces a 256-bit digest." (p.24)
- [19]: "Ascon-XOF128 is a XOF, where the output size of the message hash can be selected by the user." (p.1)
- [20]: "Ascon-XOF128 accepts an additional input $L > 0$, which specifies the desired output length in bits." (p.25)

Bibliography (3/6)

- [21]: "Ascon-CX0F128 is a customised XOF... This feature enables domain separation, ensuring that two instances of Ascon-CX0F128 with the same input message but different customisation strings produce distinct outputs." (p.28)
- [22]: "The Ascon family comprises a suite of cryptographic primitives that provide authenticated encryption... a hash function, and extendable output function (XOF) capabilities." (p.i), "The same permutations are used... to share logic and, therefore, have a more compact implementation." (p.2)
- [23]: "Hash and XOF algorithms are based on the Ascon-p[12] permutation in a sponge-based mode. ... This mode comprises three main steps: initialisation, absorbing the message, and squeezing the output." (p.23)
- [24]: (p.23)
- [25]: "The permutations follow the Substitution-Permutation-Network (SPN) structure... Note that Ascon-p[8] and Ascon-p[12] are the main building blocks of the Ascon family" (p.8)
- [26]: "The permutations operate on the 320-bit state S ..." (p.8)
- [27]: "...which is represented as five 64-bit words denoted as S_i for $0 \leq i \leq 4$ " (p. 8)
- [28]: "Ascon-AEAD128 is a nonce-based AEAD scheme, offering 128-bit security strength..." (p.1)
- [29]: "Note that Ascon-p[8] and Ascon-p[12] are the main building blocks of the Ascon family" (p. 8)
- [30]: "Ascon-AEAD128 is a nonce-based AEAD scheme, offering 128-bit security strength in the single-key setting." (p.1), "...provides 128-bit security strength... for the confidentiality of the plaintext... and integrity" (p.20)

Bibliography (4/6)

[31]: "However, these algorithms are designed to provide some resilience against unintentional nonce reuse." (p.21)

[32]: "R6. Data limit. The total amount of data processed during encryption and decryption, including the nonce, shall not exceed 2^{54} bytes for a given key." (p.19)

[33]: "In 2023, the National Institute of Standards and Technology (NIST) announced the selection of the Ascon family... to provide efficient cryptographic solutions for resource-constrained devices." (p.i), "The family is developed to offer a viable alternative when the Advanced Encryption Standard (AES) may not perform optimally." (p.i)

[34]: "Multiple functionalities. The same permutations are used to construct multiple functionalities... to share logic and, therefore, achieve a more compact implementation." (p.2), "Inverse-free. Since all members of the Ascon family only use the underlying permutations in the forward direction, there is no need to implement the inverse permutations." (p.2)

[35]: "The Ascon family... provides robust security, efficiency and flexibility, making it ideal for resource-constrained environments" (p.i), "Table 9. Security strengths...128" (p.31)

[36]: Table 8 (p. 21)

[37]: Table 9 (p. 31)

[38]: "Initialisation. ... the 320-bit internal state S is initialised as the concatenation of IV , K , and N ... Next, S is updated using the permutation $Ascon-p[12]$ " (p.13)

[39]: "Processing associated data. ... Each associated data block... is incorporated into the first 128 bits of the state... and the permutation $Ascon-p[8]$ is applied... to update the state with a constant... that provides domain separation." (p.13)

[40]: "Processing plaintext. ... For each P_i ... the state S is updated... generating the corresponding ciphertext block C_i ... and the permutation $Ascon-p[8]$ is applied to update the state" (p.14)

Bibliography (5/6)

- [41]: "Finalisation and tag generation. ... the key is first loaded into state S ... and state S is then updated using the Ascon-p[12] permutation... Finally, the tag T is generated by XORing the key" (p.14)
- [42]: "Decryption in Ascon-AEAD128 consists of four phases: initialisation, associated data processing, ciphertext processing, and finalisation." (p.15)
- [43]: "Processing the ciphertext. ... $P_i \leftarrow S[0:127] \text{ XOR } C_i$... $S[0:127] \leftarrow C_i$... and the permutation Ascon-p[8] is applied" (p.17)
- [44]: "Finalisation. ... $T' \leftarrow S[192:319] \text{ XOR } K$... As the final step, the computed T' is compared with the input T . If the two match, the plaintext P is returned. Otherwise, an error message 'fail' is returned." (p.18)
- [48]: "When (N, A) pairs are distinct... the guarantees assume... that any given nonce is reused for encryption with the same key no more than times." (p.21)
- [49]: "Table 7. Security strength... in the u -key setting... $\min\{128 - \log_2 u, \lambda\}$ " (p.21)

Dobraunig, C. et al. (2021) 'ASCON v1.2: Lightweight Authenticated Encryption and Hashing', Journal of Cryptology, 34(3):
<https://doi.org/10.1007/s00145-021-09398-9>:

This is the academic paper (Journal Paper) published by the creators of ASCON in the scientific journal *Journal of Cryptology* in 2021. It was used to verify the previous source.

Bibliography (6/6)

Lefevre, C. and Mennink, B. (2024) Generic security of the AsCon mode: on the power of key blinding:
<https://eprint.iacr.org/2023/796:>

[45]: “We prove that AsCon satisfies this security property, thanks to its unique key-blinding technique.” (Abstract, p. 1)

[46]: “The most crucial difference is the key blinding at the beginning and at the end to achieve extra robustness against state recovery.” (Introduction, p. 2)

[47]: “The core idea behind why AsCon achieves authenticity under state recovery... is that the calls to the outer permutation p are masked by K on both sides. This means that, even if the adversary learns all intermediate states, it cannot ‘pass through’ the permutations p .” (Section 5.4.2, p.17)

Dobraunig, C., Eichlseder, M., Mendel, F., & Schl  ffer, M. (n.d.). ASCON – Lightweight Cryptography. Retrieved from
<https://ascon.isec.tugraz.at/index.html:>

Official website of the Ascon project, maintained by the research team at TU Graz. It was used to understand the functioning of hash and AEAD algorithms, as well as to verify technical details such as the ‘Inverse-Free’ property in the algorithm’s implementation.